

Google Antigravity Customization Guide

Creating a Custom Skill in .agents/skills/gemma-eval/SKILL.md

Overview: In Google Antigravity, a **Skill** is a self-contained, modular package of operational runbooks, helper scripts, and reference manuals. Skills use **Progressive Disclosure**: the agent only indexes the YAML frontmatter (name, description) at session start, and dynamically loads the Markdown body and executable scripts only when the user prompt matches the skill's trigger intent.

1. Custom Skill Directory Structure

A well-architected Antigravity skill follows a modular folder hierarchy under .agents/skills/<skill-name>/:

```
.agents/skills/gemma-eval/
■■■ SKILL.md           # Required: Main instruction file with YAML frontmatter
■■■ scripts/           # Optional: Executable automation scripts
■   ■■■ verify_environment.py # Pre-flight environment & GPU verification
■■■ references/         # Optional: In-depth technical reference documentation
    ■■■ metrics_reference.md # Detailed benchmark thresholds and baseline scores
```

2. YAML Frontmatter Anatomy (The Trigger Contract)

The YAML frontmatter block at the top of SKILL.md is the most critical component for skill discovery:

Field	Type	Requirement & Behavioral Purpose
name	String	Required. Unique, lowercase, hyphenated identifier (e.g. gemma-eval). Used for logging and explicit skill invocation.
description	String (YAML folded / multiline)	Required (The Trigger Contract). The agent inspects this description to decide when to activate the skill. Must explicitly describe <i>what</i> the skill does and <i>when</i> to invoke it in third-person.

3. Full SKILL.md Implementation

Below is the exact production-ready implementation created in .agents/skills/gemma-eval/SKILL.md:

```

---
name: gemma-eval
description: >-
  Use this skill when the user asks to evaluate, benchmark, fine-tune, or test
  Gemma models (e.g., Gemma 2B, 7B, 9B, 27B), run automated model evaluations,
  compute perplexity/loss metrics, or validate inference prompt templates.
---

# Gemma Model Evaluation & Benchmarking Runbook

## 1. Environment & Pre-Flight Check
- Run the pre-flight verification script:
  [verify_environment.py](./scripts/verify_environment.py)
- Confirm model weights are accessible locally or in Hugging Face cache.

## 2. Running Automated Evaluation
```bash
python .agents/skills/gemma-eval/scripts/evaluate_model.py \
 --model-id "google/gemma-2-9b-it" \
 --benchmark "mmlu-pro" --batch-size 4 --output-dir "./eval_results"
```

## 3. Metric Inspection & Thresholds
- Review `./eval_results/metrics.json` against baseline thresholds:
  [metrics_reference.md](./references/metrics_reference.md)

## 4. Verification & Reporting
1. Verify `metrics.json` and `eval_summary.md` are generated.
2. Confirm no OOM warnings or quantization degradation occurred.

```

4. Architectural Best Practices for Custom Skills

- **Progressive Disclosure:** Keep SKILL.md concise. Move verbose documentation (e.g. detailed metric matrices, API guides) into references/ and link to them using relative Markdown links.
- **Encapsulated Executable Scripts:** Wrap complex command pipelines inside scripts/ utilities rather than pasting massive multi-line shell scripts into the runbook.
- **Clear Verification Steps:** Every skill must provide explicit validation criteria (e.g., verifying log files, return codes, output JSON structures) so the agent can self-verify execution.
- **No General Coding Fluff:** Avoid explaining basic syntax or concepts the LLM already understands; focus strictly on project-specific procedures.

Implementation Status: The gemma-eval skill structure has been created in your workspace at `.agents/skills/gemma-eval/`, complete with `SKILL.md`, `scripts/verify_environment.py`, and `references/metrics_reference.md`. It is now active and ready for automatic discovery by Antigravity.